



## Table of Contents

|         |                                     |                                              |
|---------|-------------------------------------|----------------------------------------------|
| Phase 1 | Prepare Your Codebase               | <i>Deploy-ready configuration</i>            |
| Phase 2 | Terraform — Full AWS Infrastructure | <i>VPC, EC2, Security Groups, RDS, Redis</i> |
| Phase 3 | Deploy JARVIS on EC2                | <i>Docker Compose production deployment</i>  |
| Phase 4 | Domain + SSL Setup                  | <i>Professional domain, HTTPS, Nginx</i>     |
| Phase 5 | CI/CD Automation                    | <i>GitHub Actions — zero-touch deploys</i>   |
| Phase 6 | Monitoring + Health                 | <i>Grafana, Prometheus, alerts</i>           |
| Bonus   | Go-Live Checklist                   | <i>Final verification before clients</i>     |
| Future  | Enterprise Upgrade Path             | <i>ECS Fargate migration roadmap</i>         |

## Where You Are Right Now

---

|                |                                    |
|----------------|------------------------------------|
| ■ EC2 Instance | Live Ubuntu server on AWS          |
| ■ Docker       | Installed and running              |
| ■ Terraform    | Installed, AWS provider configured |
| ■ AWS CLI      | Configured with credentials        |
| ■ Nginx        | Deployed publicly                  |
| ■ Ports Open   | 22, 80, 443, 3000, 8080            |
| ■ JARVIS Code  | Not yet deployed to server         |
| ■ Domain + SSL | Not yet configured                 |
| ■ CI/CD        | Not yet automated                  |
| ■ Monitoring   | Not yet running                    |

By end of this guide: Everything marked ■ will be ■ and JARVIS will be fully live.

## 1

## Prepare Your Codebase

Configure JARVIS for production deployment

What this phase does: Makes your JARVIS codebase deployment-ready before it touches the server.

### 1 Create production .env file

On your local machine, create a .env file with all required keys:

```
# On your local machine
cd devops-docker-project
cp .env.example .env

# Edit .env and fill in these values:
DATABASE_URL=postgresql+asyncpg://jarvis:yourpassword@postgres:5432/jarvis
REDIS_URL=redis://redis:6379/0
SECRET_KEY=your-super-secret-key-minimum-32-chars
ANTHROPIC_API_KEY=sk-ant-...
OPENAI_API_KEY=sk-...
STRIPE_SECRET_KEY=sk_live_...
SLACK_WEBHOOK_URL=https://hooks.slack.com/...
APP_BASE_URL=https://yourdomain.com
DEBUG=false
ENVIRONMENT=production
```

### 2 Verify Docker Compose works locally

```
# Test locally first
cd infrastructure
docker compose up -d
docker compose ps

# All services should show: healthy or running
curl http://localhost:8000/health

# Expected: {"status": "ok", "system": "JARVIS"}
```

### 3 Push everything to GitHub

```
git add .
git commit -m 'feat: production-ready configuration'
git push origin claude/jarvis-cans-api-integration-ZThTD

# NEVER push .env — it is in .gitignore
```

■ ■ Never commit your .env file. It contains secrets. It is already in .gitignore.

## 2

## Terraform — Full AWS Infrastructure

Automate your entire cloud setup with one command

What this phase does: Uses Terraform to provision VPC, EC2, Security Groups, Elastic IP, and all networking automatically.

### 4 Navigate to Terraform directory

```
cd infra/terraform
ls
# You should see: main.tf, variables.tf, outputs.tf
```

### 5 Update Terraform variables

Edit terraform.tfvars (create if not exists):

```
# infra/terraform/terraform.tfvars
region          = "ap-south-2"
instance_type   = "t3.medium"
key_name        = "your-ec2-keypair-name"
db_password     = "your-secure-db-password"
project_name    = "jarvis-production"
```

### 6 Run Terraform to provision infrastructure

```
terraform init
# Downloads AWS provider

terraform plan
# Shows exactly what will be created – review carefully

terraform apply
# Type 'yes' when prompted
# Takes 3-5 minutes

terraform output
# Copy the EC2 public IP – you will need it
```

### 7 Save your EC2 IP and SSH key

```
# Terraform outputs your new server IP
# Save it – example: 13.235.45.112

# Test SSH connection
ssh -i your-key.pem ubuntu@YOUR_EC2_IP
# You should see Ubuntu welcome message
```

|                  |                                              |
|------------------|----------------------------------------------|
| VPC              | Isolated network for your infrastructure     |
| Public Subnet    | Where EC2 runs                               |
| Internet Gateway | Connects your VPC to the internet            |
| Security Group   | Firewall — ports 22, 80, 443, 8000 open      |
| EC2 t3.medium    | Your production server (2 CPU, 4GB RAM)      |
| Elastic IP       | Static IP — never changes even after restart |
| Key Pair         | SSH access credentials                       |

## 3

## Deploy JARVIS on EC2

Get the full system running on your server

What this phase does: SSHs into your EC2, installs dependencies, clones JARVIS, and starts all services.

### 8 SSH into your EC2 server



```
ssh -i your-key.pem ubuntu@YOUR_EC2_IP
```

### 9 Install Docker on the server



```
# Update packages
sudo apt update && sudo apt upgrade -y

# Install Docker
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh

# Add ubuntu user to docker group
sudo usermod -aG docker ubuntu
newgrp docker

# Install Docker Compose
sudo apt install docker-compose-plugin -y

# Verify
docker --version
docker compose version
```

### 10 Clone JARVIS codebase



```
# Clone your repository
git clone https://github.com/syedabrarfff-stack/devops-docker-project.git
cd devops-docker-project
```

### 11 Create .env on the server



```
# Create .env directly on server
nano .env

# Paste all your environment variables
# Save: Ctrl+X → Y → Enter

# Verify it saved
cat .env | head -5
```

## 12 Start JARVIS — one command

```
# Navigate to infrastructure
cd infrastructure

# Start everything
docker compose up -d

# Watch services start
docker compose ps

# Check backend health
curl http://localhost:8000/health
# Expected: {"status": "ok", "system": "JARVIS"}

# Check deep readiness
curl http://localhost:8000/readyz
```

## 13 Verify all services running

```
docker compose ps
# Expected output:
# jarvis_postgres    running (healthy)
# jarvis_redis        running (healthy)
# jarvis_backend      running (healthy)
# jarvis_frontend     running
# jarvis_nginx        running
```



## 4

## Domain + SSL Setup

Make JARVIS accessible at your professional domain

What this phase does: Points your domain to EC2, configures Nginx, and installs free SSL certificate.

### 14 Buy your domain

Recommended: Namecheap.com — search for aliyarsolutions.com (~\$10/year)

### 15 Point domain to EC2 IP

In Namecheap DNS settings, add these records:

| Type     | Host | Value       | TTL  |
|----------|------|-------------|------|
| A Record | @    | YOUR_EC2_IP | Auto |
| A Record | www  | YOUR_EC2_IP | Auto |
| A Record | api  | YOUR_EC2_IP | Auto |

*Note: DNS propagation takes 5-30 minutes. Test with: ping yourdomain.com*

### 16 Install SSL certificate (free — Certbot)

```

# Install Certbot
sudo apt install certbot python3-certbot-nginx -y

# Get SSL certificate (replace with your domain)
sudo certbot --nginx -d aliyarsolutions.com -d www.aliyarsolutions.com

# Follow prompts:
# Enter email address
# Agree to terms: Y
# Share email with EFF: N

# Certbot automatically configures Nginx for HTTPS
# Certificate auto-renews every 90 days

# Test renewal
sudo certbot renew --dry-run

```

### 17 Update Nginx config for your domain



```
# Edit Nginx config
sudo nano /etc/nginx/conf.d/default.conf

# OR update infrastructure/nginx/nginx.conf
# Replace: server_name _;
# With:     server_name aliyarsolutions.com www.aliyarsolutions.com;

# Test config
sudo nginx -t

# Reload Nginx
sudo nginx -s reload

# Test HTTPS
curl https://aliyarsolutions.com/health
```

## 5

## CI/CD Automation

Every git push deploys automatically — zero manual work

What this phase does: Connects GitHub Actions to your EC2 server. Every push to main branch auto-deploys JARVIS.

## 18 Generate SSH deploy key on EC2

```
# On your EC2 server
ssh-keygen -t ed25519 -C 'jarvis-deploy' -f ~/.ssh/deploy_key -N ''

# Add public key to authorized_keys
cat ~/.ssh/deploy_key.pub >> ~/.ssh/authorized_keys

# Copy the PRIVATE key – you need it for GitHub
cat ~/.ssh/deploy_key

# Copy everything including -----BEGIN... and -----END...
```

## 19 Add secrets to GitHub repository

Go to: GitHub → Your Repo → Settings → Secrets → Actions → New Repository Secret

| Secret Name       | Value                                |
|-------------------|--------------------------------------|
| EC2_SSH_KEY       | Paste the private key from Step 18   |
| EC2_HOST          | Your EC2 public IP or domain         |
| EC2_USER          | ubuntu                               |
| EC2_DEPLOY_PATH   | /home/ubuntu/devops-docker-project   |
| SLACK_WEBHOOK_URL | Your Slack webhook for notifications |
| APP_BASE_URL      | https://aliyarsolutions.com          |

## 20 Create GitHub Actions deploy workflow

Create `.github/workflows/ec2-deploy.yml`:

```
name: Deploy JARVIS to EC2
on:
  push:
    branches: [main]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Deploy to EC2
        uses: appleboy/ssh-action@v1.0.0
        with:
          host: ${ secrets.EC2_HOST }
          username: ${ secrets.EC2_USER }
          key: ${ secrets.EC2_SSH_KEY }
          script: |
            cd ${ secrets.EC2_DEPLOY_PATH }
            git pull origin main
            cd infrastructure
            docker compose pull
            docker compose up -d --build
            curl -f http://localhost:8000/health
```

## 21 Test the automation

```
# On local machine – make any small change
echo '# deployed' >> README.md
git add . && git commit -m 'test: CI/CD automation'
git push origin main

# Go to GitHub → Actions tab
# Watch the deployment run automatically
# Green checkmark = success

# From this point: every push = automatic deploy
```

## 6

## Monitoring + Health

Real-time visibility into JARVIS operations

What this phase does: Sets up Prometheus metrics collection and Grafana dashboards for full operational visibility.

### 22 Add monitoring to Docker Compose

```
# Add to infrastructure/docker-compose.yml:

prometheus:
  image: prom/prometheus:latest
  container_name: jarvis_prometheus
  volumes:
    - ./prometheus.yml:/etc/prometheus/prometheus.yml
  ports:
    - '9090:9090'
  networks:
    - jarvis_net

grafana:
  image: grafana/grafana:latest
  container_name: jarvis_grafana
  ports:
    - '3001:3000'
  environment:
    - GF_SECURITY_ADMIN_PASSWORD=yourpassword
  networks:
    - jarvis_net
```

### 23 Set up automated health alerts

```
# Install Watchtower – auto-pulls new Docker images
docker run -d --name watchtower \
  -v /var/run/docker.sock:/var/run/docker.sock \
  containrrr/watchtower \
  --interval 300
# Checks for updates every 5 minutes

# Set up cron for health monitoring
crontab -e
# Add: */5 * * * * curl -f http://localhost:8000/health || ...
# This pings health every 5 minutes
```

### 24 Configure uptime monitoring (free)

Use UptimeRobot (free) to monitor your domain 24/7 and alert you via Slack/email if down:

1. Go to [uptimerobot.com](https://uptimerobot.com) — free account
2. Add Monitor → HTTP(s) → <https://aliyarsolutions.com/health>

3. Set interval: every 5 minutes
4. Add Slack notification → your JARVIS Slack webhook
5. You will be alerted within 5 minutes of any downtime



## Go-Live Checklist

Verify everything before your first client

- EC2 running and accessible via SSH
  - All Docker containers healthy (docker compose ps)
  - Elastic IP attached (IP never changes)
  - Security groups configured correctly
- 
- GET /health returns {"status": "ok"}
  - GET /readyz returns all subsystems healthy
  - Frontend loads at http://YOUR\_IP:3000
  - JARVIS dashboard fully functional
  - Team registry seeded (9 members visible)
  - Service catalog seeded (30 divisions)
- 
- Domain points to EC2 IP
  - https://yourdomain.com loads correctly
  - SSL certificate valid (green padlock in browser)
  - www.yourdomain.com redirects to yourdomain.com
- 
- GitHub Actions workflow runs on push to main
  - Test deploy completes successfully
  - Health check passes after deploy
  - Slack notification received on deploy
- 
- .env file NOT in git repository
  - All API keys set as environment variables
  - SSH access only via key (password disabled)
  - Ports 3000, 8000 blocked externally (Nginx proxies them)
- 
- UptimeRobot monitoring active
  - Slack alerts configured
  - Grafana dashboard accessible
  - Daily briefing job running (check at 07:00)

→

## Enterprise Upgrade Path

After first client — migrate to full AWS managed services

Once you have revenue, upgrade to the enterprise architecture. All Terraform is already written in `infra/terraform/`.

| Current (EC2)        | Enterprise (ECS Fargate)   | Benefit                    |
|----------------------|----------------------------|----------------------------|
| EC2 t3.medium        | ECS Fargate tasks          | No server management       |
| Docker Compose       | ECS Task Definitions       | Auto-scaling               |
| PostgreSQL in Docker | AWS RDS PostgreSQL         | Auto-backup, multi-AZ      |
| Redis in Docker      | AWS ElastiCache            | Managed, highly available  |
| Nginx on EC2         | AWS ALB                    | Auto-scaling load balancer |
| Certbot SSL          | AWS ACM                    | Auto-renewing, zero config |
| .env file            | AWS Secrets Manager        | Encrypted, audited access  |
| GitHub Actions → SSH | GitHub Actions → ECR → ECS | Blue/green zero-downtime   |
| ~\$20/month          | ~\$130-140/month           | Enterprise grade           |

```
cd infra/terraform
terraform init
terraform plan
terraform apply

# Entire enterprise infrastructure provisioned in ~10 minutes
# ECS Fargate + RDS + ElastiCache + ALB + ACM + Secrets Manager
```

| Phase            | What You Get                        | Time       |
|------------------|-------------------------------------|------------|
| 1 — Prepare      | Codebase production-ready           | 30 min     |
| 2 — Terraform    | Full AWS infrastructure automated   | 1 hour     |
| 3 — Deploy       | JARVIS live on EC2                  | 1 hour     |
| 4 — Domain + SSL | Professional HTTPS domain           | 1 hour     |
| 5 — CI/CD        | Zero-touch automated deploys        | 1 hour     |
| 6 — Monitoring   | 24/7 operational visibility         | 30 min     |
| Total            | Fully live JARVIS production system | ~5-6 hours |

The infrastructure is built. The code is ready. Execute these 6 phases and JARVIS is live. — JARVIS Operations